Rendering Strategies

Contents

- Rendering polygonal objects
- Rendering parametric patch net
- Rendering CSG description

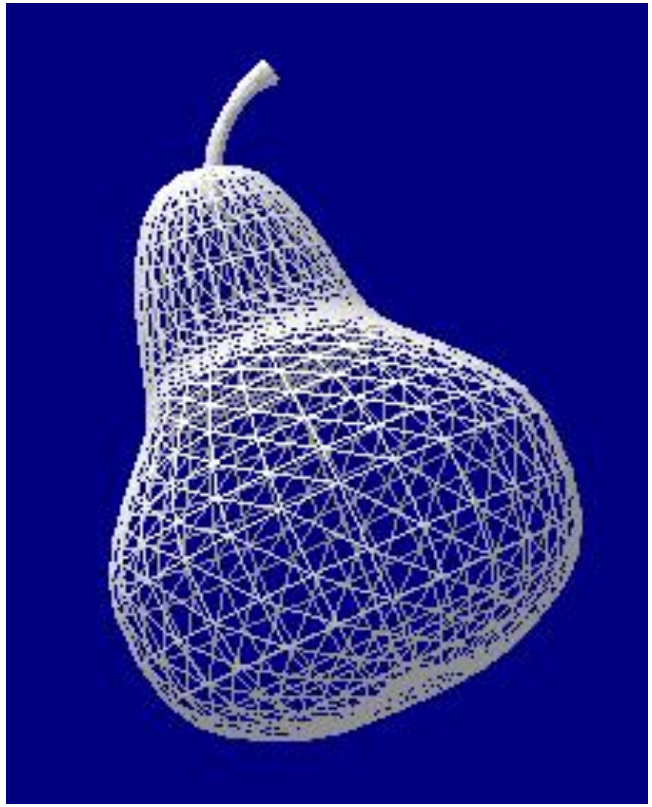
Rendering definition

- Collection of operations necessary to project a view of an object or a scene onto a view surface

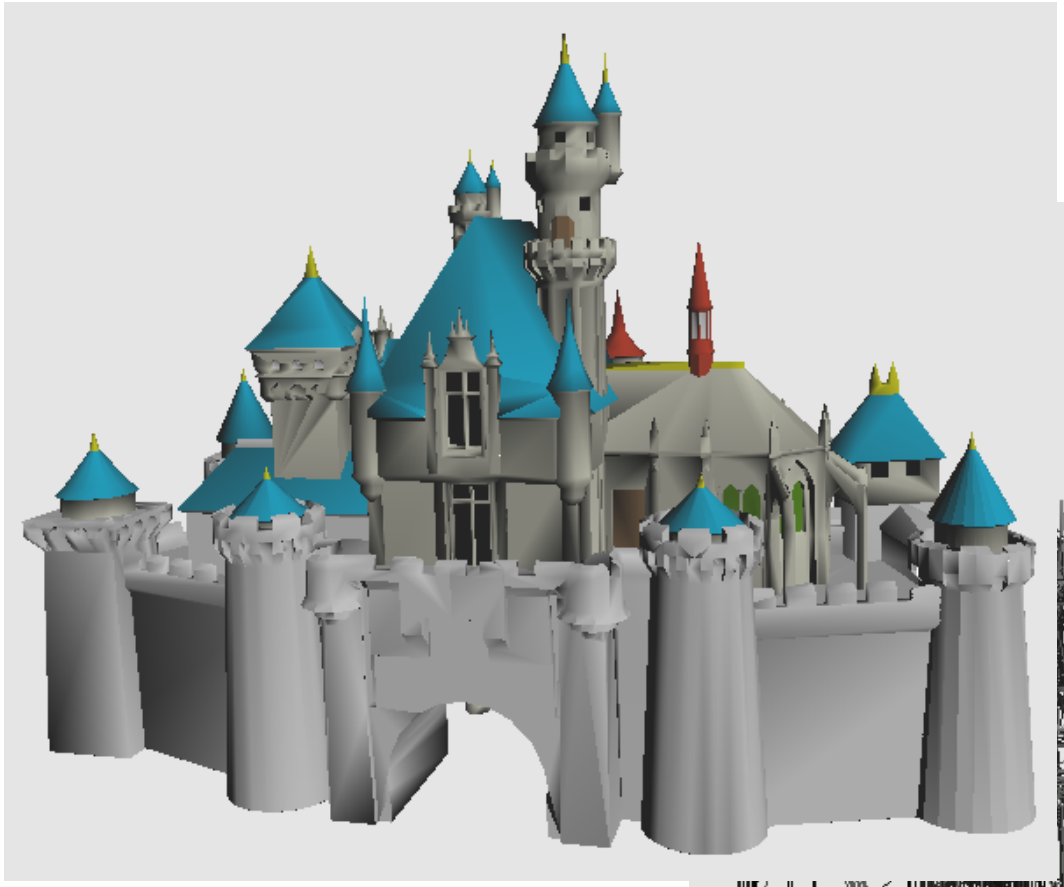
- Rendering strategies are determined by
  - Object representation
  - Algorithm options available for the internal processes

- Concerning on rendering:
  - Polygonal objects
  - Parametric patch net
  - CSG description

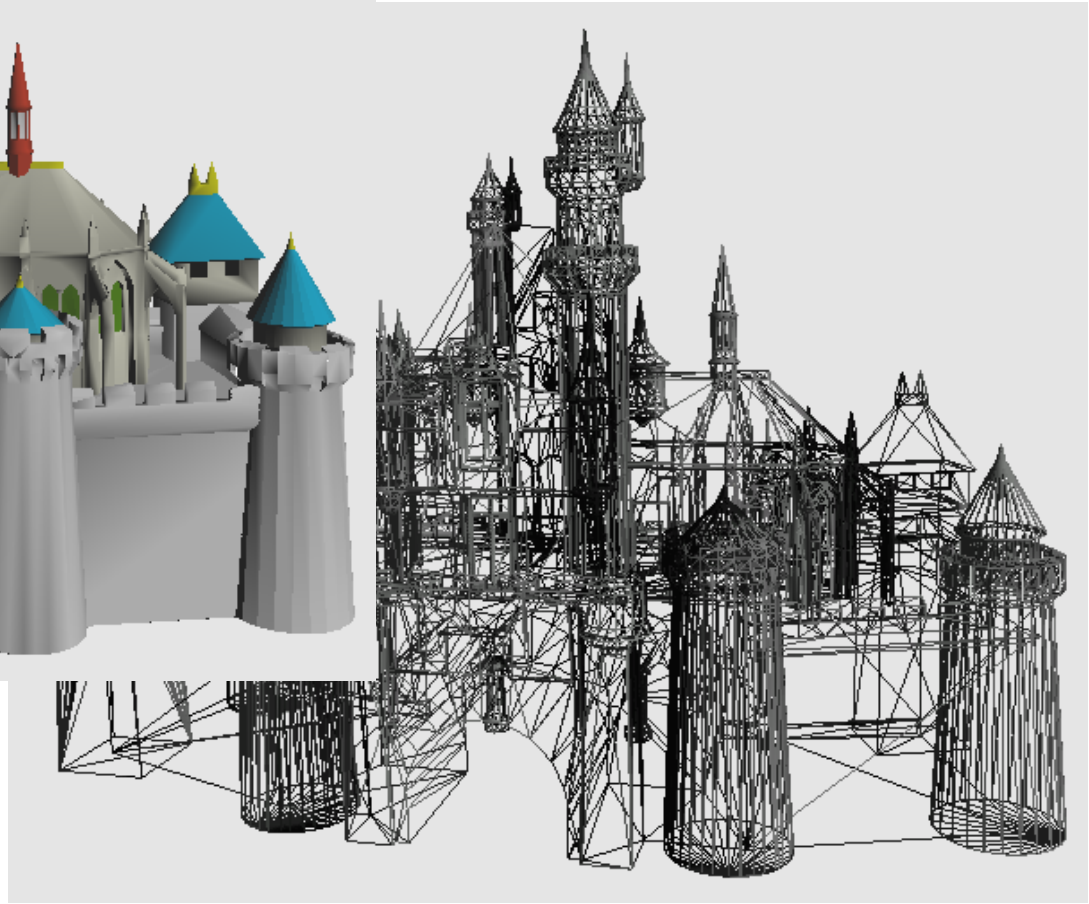
Rendering polygonal objects



Rendering polygonal objects



Number of vertices: 6620
Number of triangles: 13114
Number of normals: 16710
Face normals: 13114



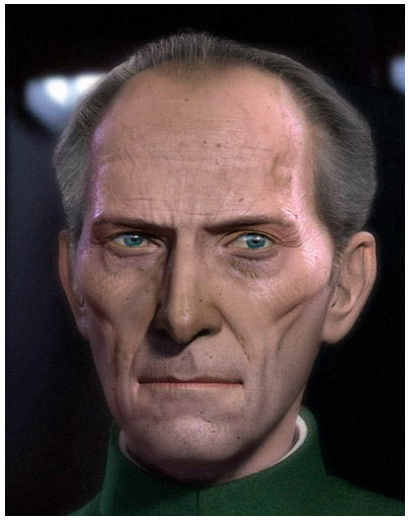
Texture coordinates: 6620
Number of materials: 9
Number of groups: 17

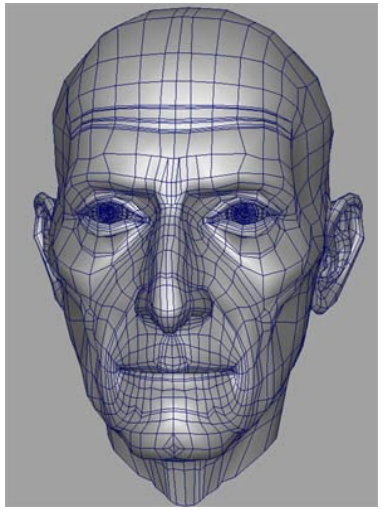
Rendering polygonal objects

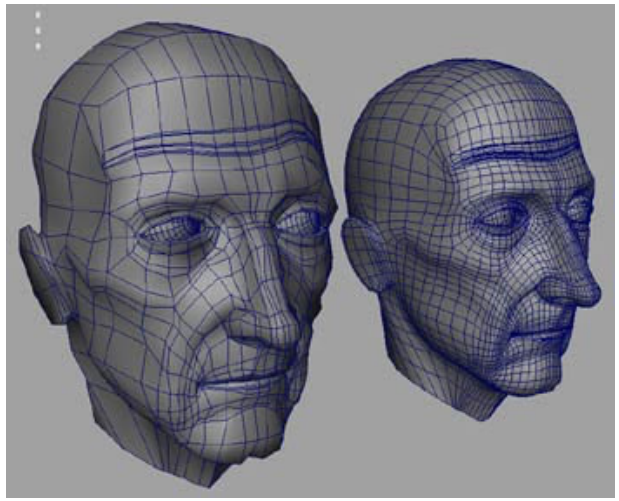
Number of vertices: 4028
Number of triangles: 3360
Number of normals: 4050
Face normals: 3360

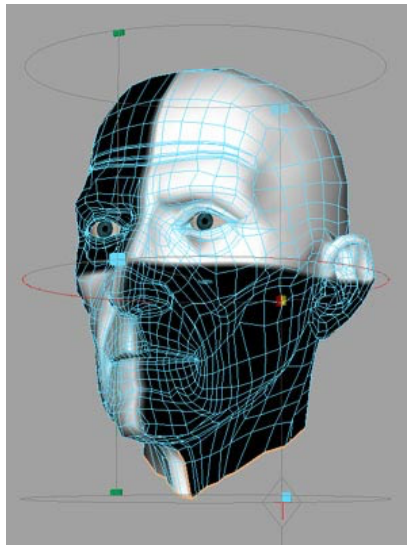
Texture coordinates: 4028
Number of materials: 7
Number of groups: 8

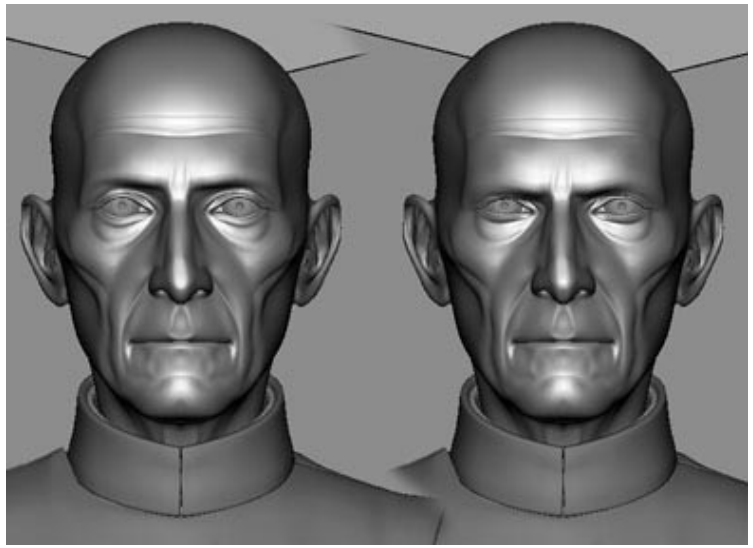
Polygonal object



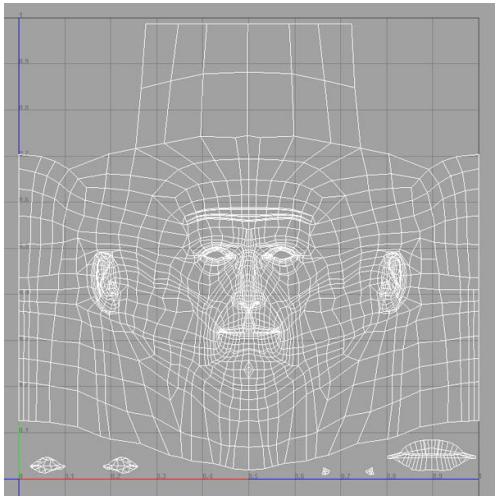
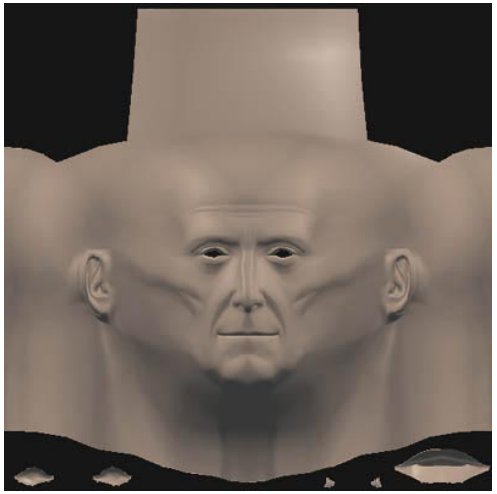


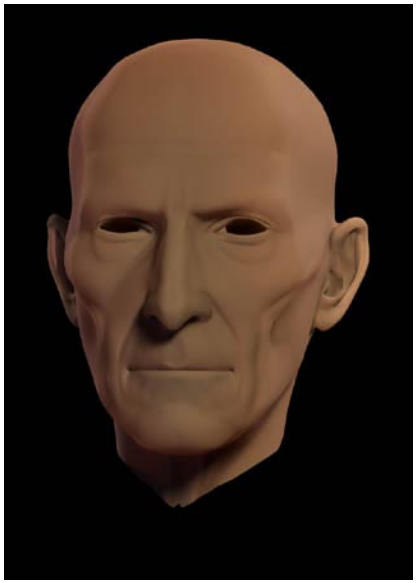






Texture mapping

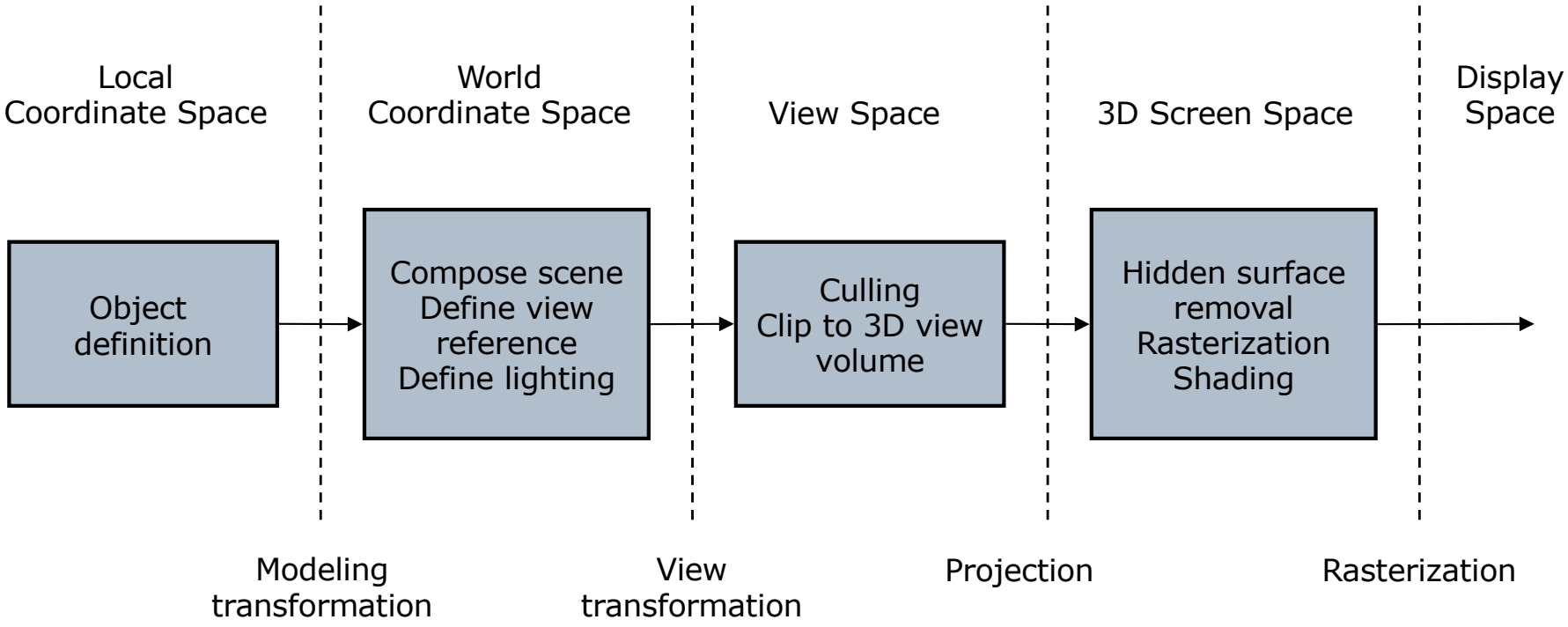



Polygonal renderer

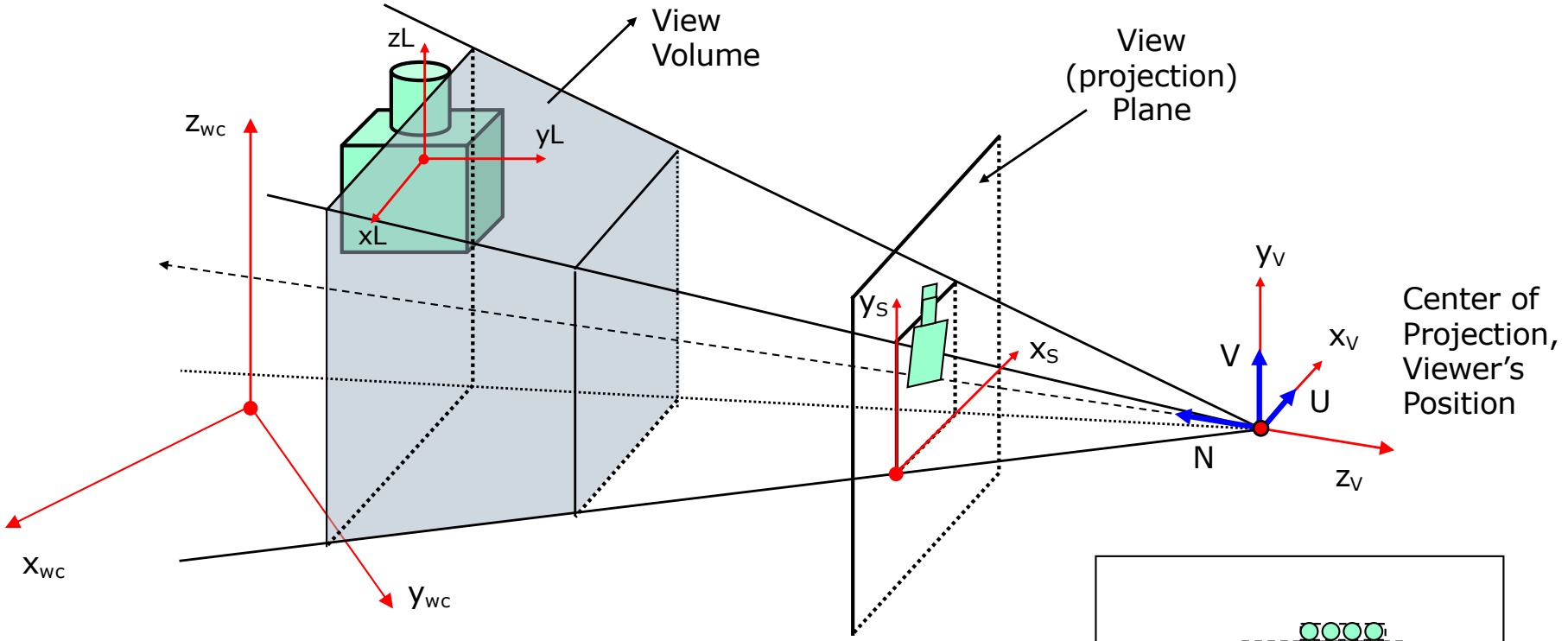
- Polygonal objects are the most common representation

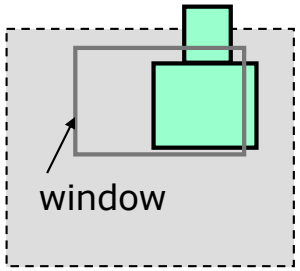
- Polygonal renderer
  - Software and hardware
  - Process a polygon as a single entity -> fast and simple processing
  - Input: list of polygons
  - Output: color for each pixel on the screen

3D Objects rendering pipeline

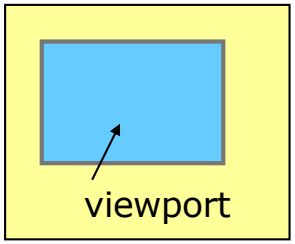


Conceptual model of 3D viewing process

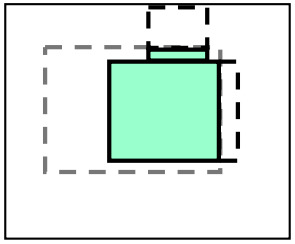




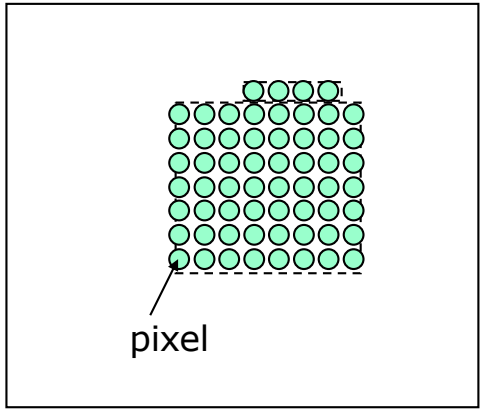
Window definition in projection plane



Viewport definition in projection plane



Window's content inside the viewport



Scan conversion onto the raster screen

Rendering pipeline

1. Local coordinate space (modeling coordinate system)
   - Operations: Object definition
   - Object stored in a database using vertex coordinates

2. World coordinate space
   - Operations: 3D transformations, scene composition, view reference setting up, lights definition

3. View space (eye coordinate system)
   - Operations:
     - culling - removes entire polygons that cannot possibly be visible from the view point
     - 3D clipping – removes the parts of the scene outside of the view volume

4. 3D screen space
   - Simultaneously operations in the innermost pixel loop:
     - Hidden surface removal - removes invisible parts of polygons
     - Shading – compares the orientation of each polygon with the light source direction and assigns a shade to the interior of the polygon
     - Rasterization – allots a value to each pixel in the projection.
     - Anti-aliasing computes intermediate values for the overlapped projected parts of the polygons

Rendering pipeline operations

- Other operations
  - Texture mapping
    - Fixed mapping – local, world, view, and screen coordinate systems
    - Viewing dependent mapping – view and screen coordinate systems
  - Shadow shape computation
    - Depends just on relative positions of objects and light sources
    - Shadow volumes, shadow polygons
    - Local, world, and view coordinate systems
  - Move viewer's position
    - Changes view volume parameters (frustum parameters)
    - World coordinate systems
  - Object transformation
    - Of positions and dimensions in world and view coordinate systems
    - Of graphics attributes (e.g. colors, textures) in local, world, view, and screen coordinate systems

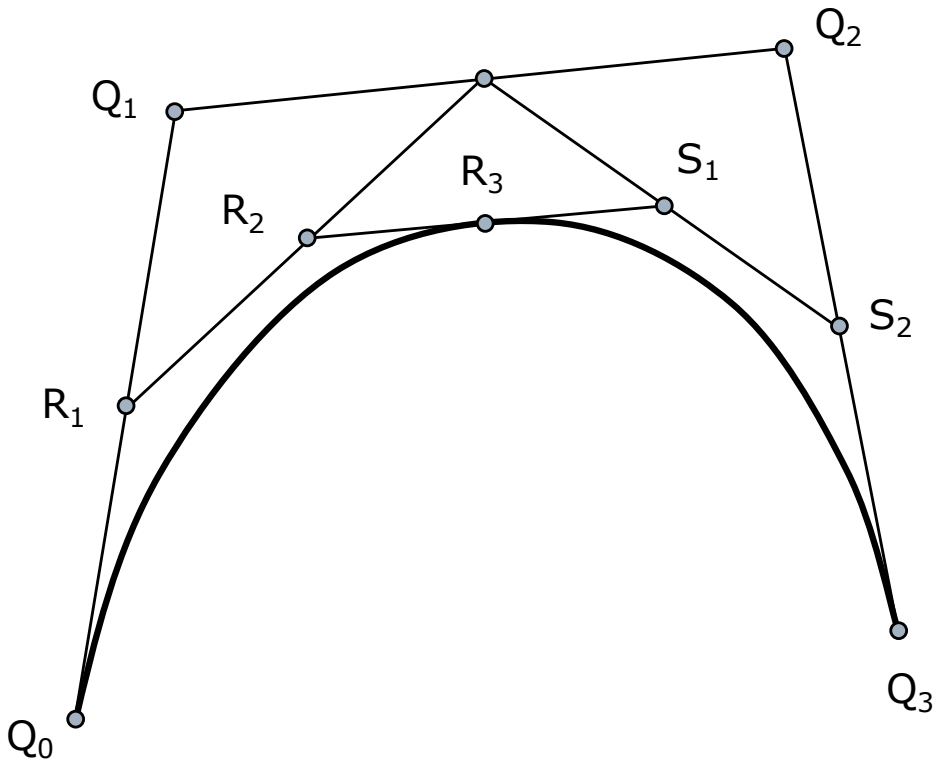
Rendering parametric patch net

- Preprocess the representation and convert the patches into planar polygons
- Original patch net provides higher accuracy representation
- Conversion is performed through a subdivision or splitting process
- Polygon size depends on the local curvature of the patch. In general the number of polygons of the converted model is grater than the number of patches in the parametric model
- Conversion provides a unification with polygon mesh rendering
- Steps of the conversion process:
  1. Basis conversion – convert the patch basis to Bézier form by matrix multiplication
  2. Patch splitting
     - subdivide by isoparametric curves (u=0.5, v=0.5) until a flatness criterion is satisfied (e.g. distance between a plane through the corner points of the patch and the patch surface)
     - De Casteljau algorithm

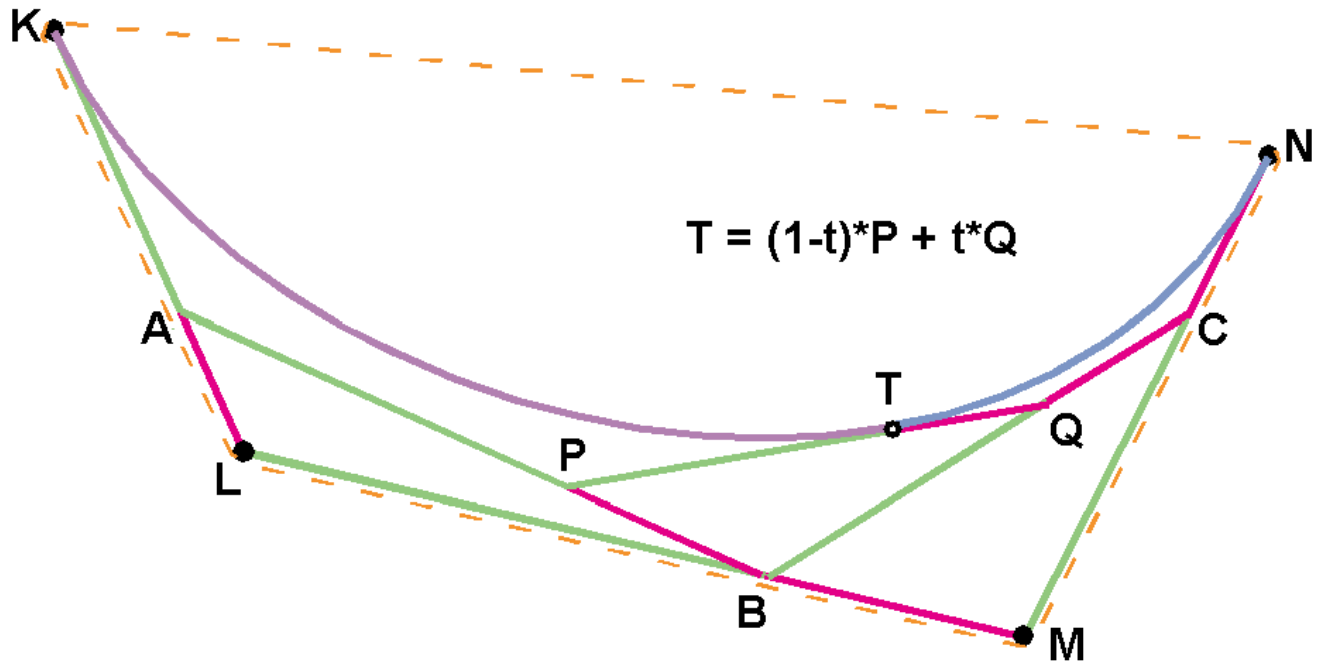
De Casteljau algorithm

$R_0 = Q_0$
$R_1 = (Q_0 + Q_1)/2$
$R_2 = R_1/2 + (Q_1 + Q_2)/4$
$R_3 = (R_2 + S_1)/2$

$S_0 = R_3$
$S_1 = (Q_1 + Q_2)/4 + S_2/2$
$S_2 = (Q_2 + Q_3)/2$
$S_3 = Q_3$



Bézier from De Casteljau



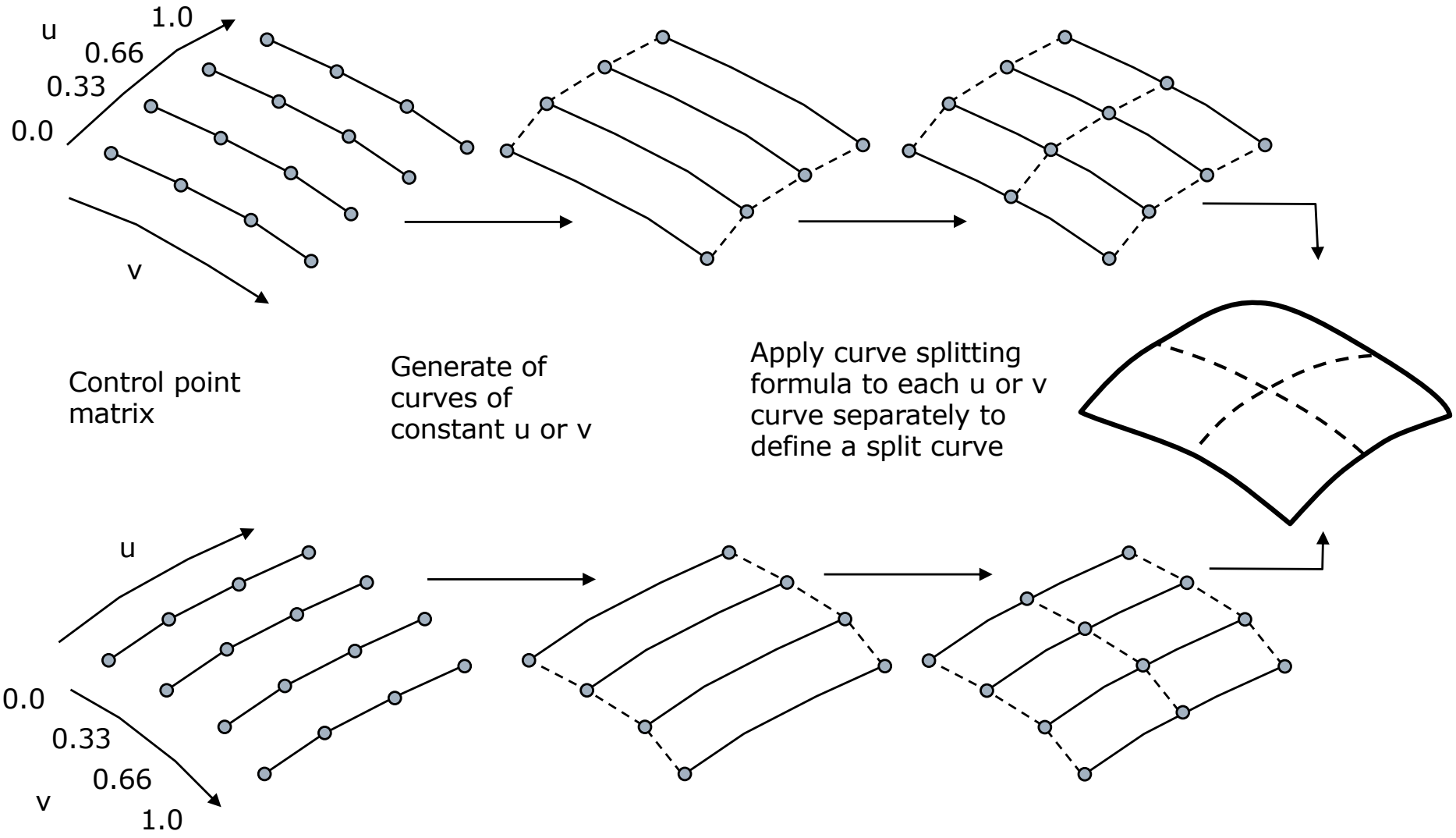
$$P = (1-t)*A + t*B \qquad Q = (1-t)*B + t*C$$

$$T = (1-t)(1-t)*A + (1-t)t*B + t(1-t)*B + t*t*C$$

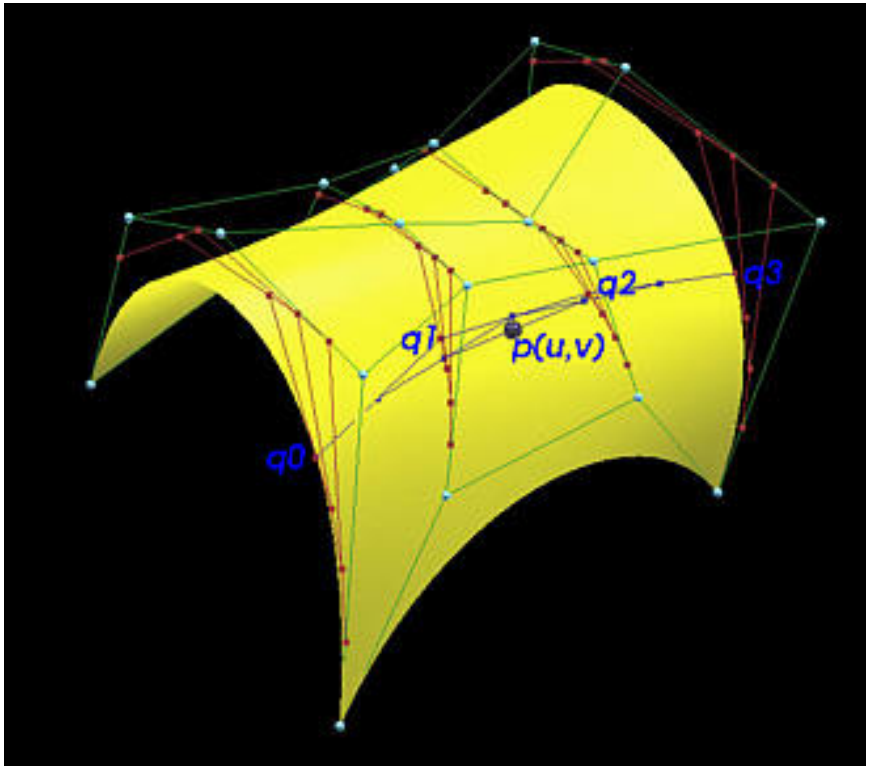
$$A = (1-t)*K + t*L \qquad B = (1-t)*L + t*M \qquad C = (1-t)*M + t*N$$

$$T = (1-t)^3*K+(1-t)^2t*L + 2(1-t)^2t*L+2(1-t)t^2*M + (1-t)t^2*M+t^3*N$$

Patch subdivision



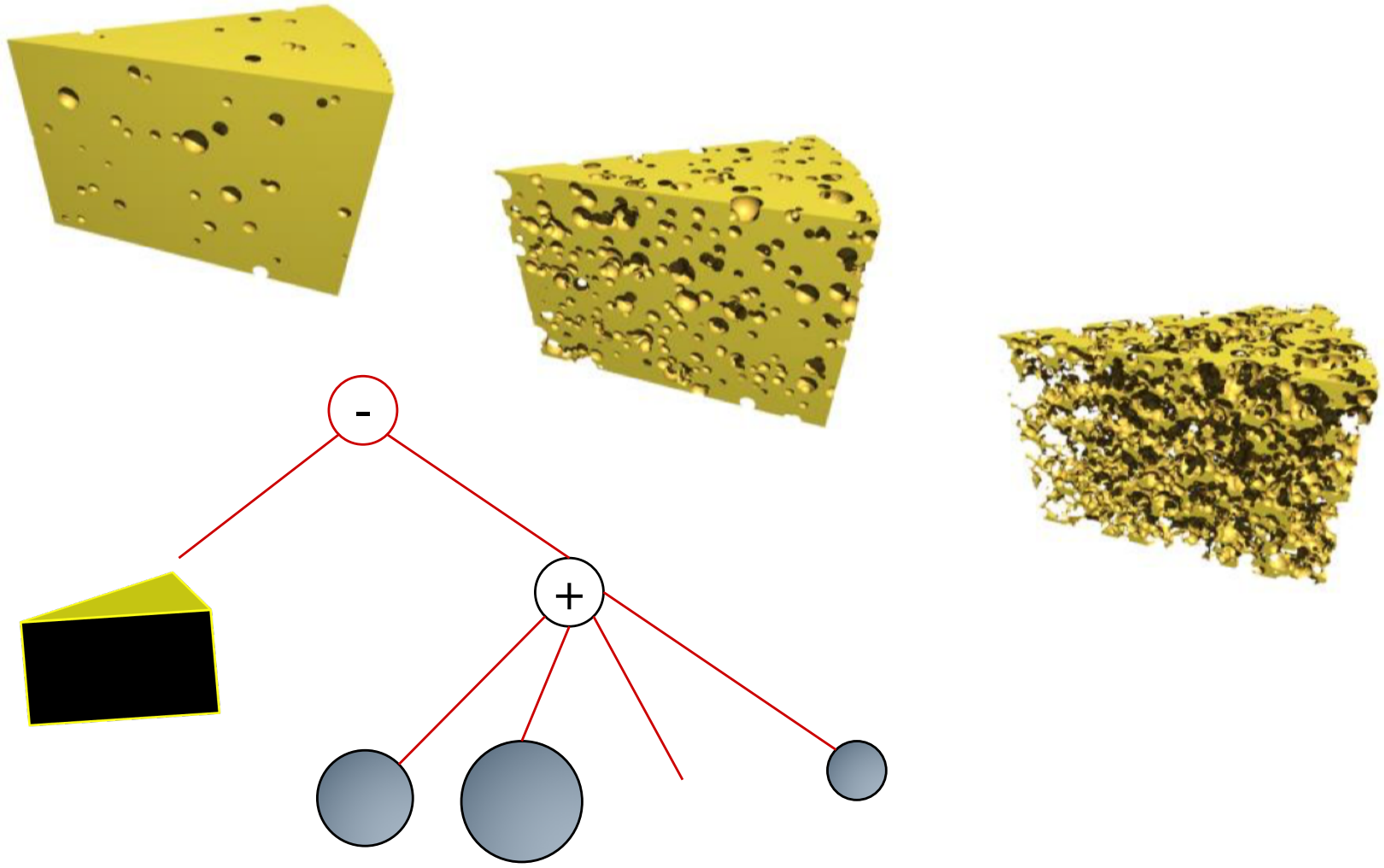
Bézier surface from De Casteljau



Rendering a CSG description

- Object database is a tree that relates a set of primitive objects to each other through boolean operations
- It is not a boundary representation (that divides the 3D space into internal and external regions of the object)
- There are 3 techniques to derive boundary representation from CSG:
  1. CSG ray tracing
  2. Conversion to a voxel representation followed by volume rendering
  3. Using a version of the Z-Buffer algorithm

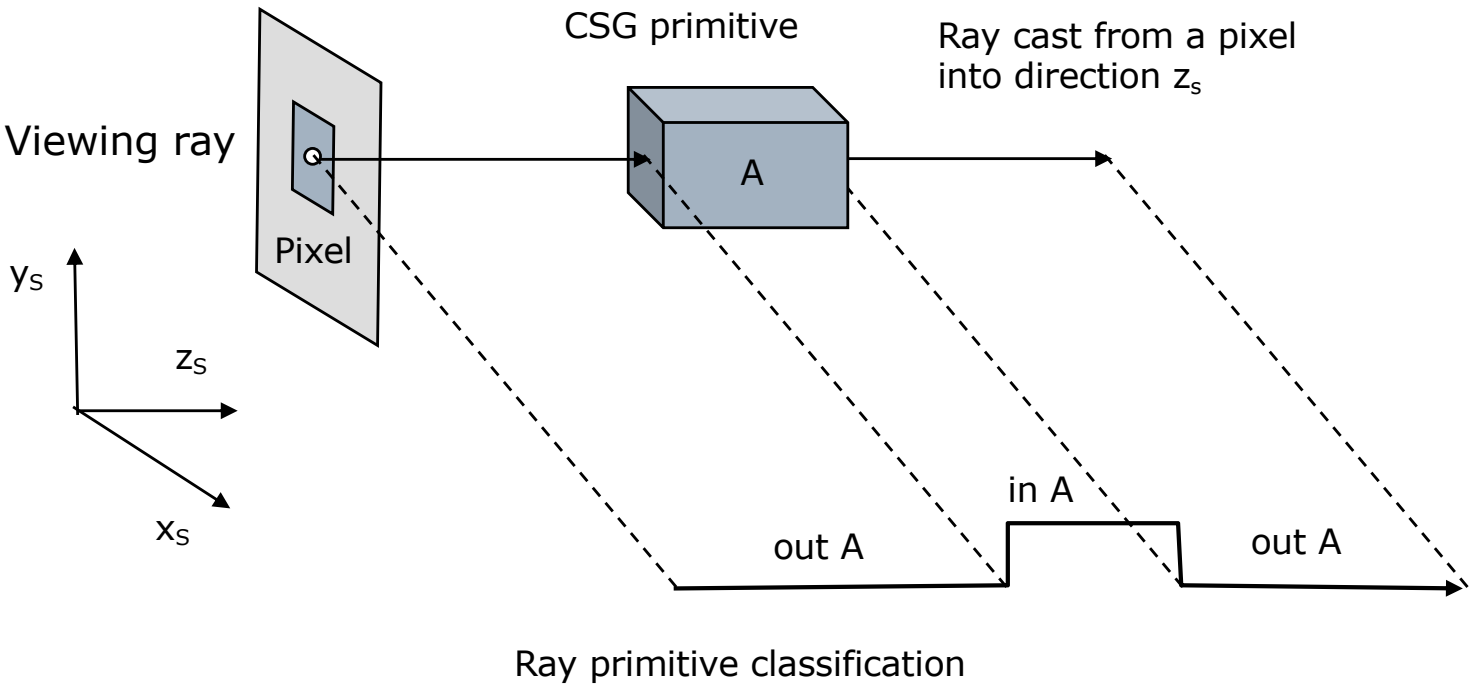
CSG Objects



CSG ray tracing

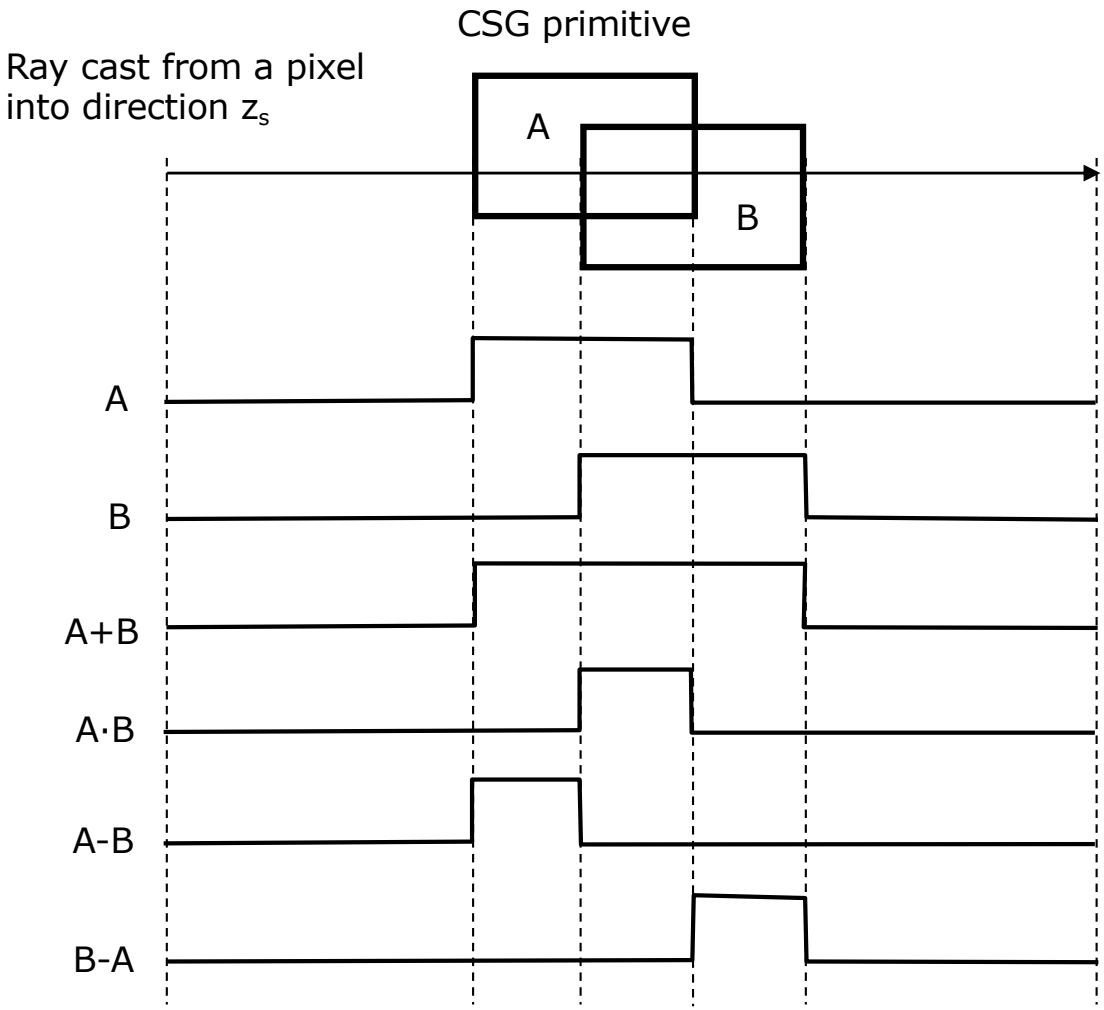
- Reduce the problem to one dimension
- Cast a ray to each pixel in the view plane
- In the simplest case (parallel projection) explore the space of the object with a set of parallel rays
- Two stages of the process:
  1. Consider a single ray.
     - Every primitive instance is compared against this ray to see if it intersects the primitive
     - Any intersections are sorted in z depth. There is a ray/primitive classification for each ray
     - Analyze any boolean combination between the first two primitives encountered along the ray
  2. Allocate a shading value to the pixel by a simple reflection model applied at the first intersection along the ray
- All operations take place in screen space, in parallel (orthogonal) projection

CSG ray tracing – primitive classification

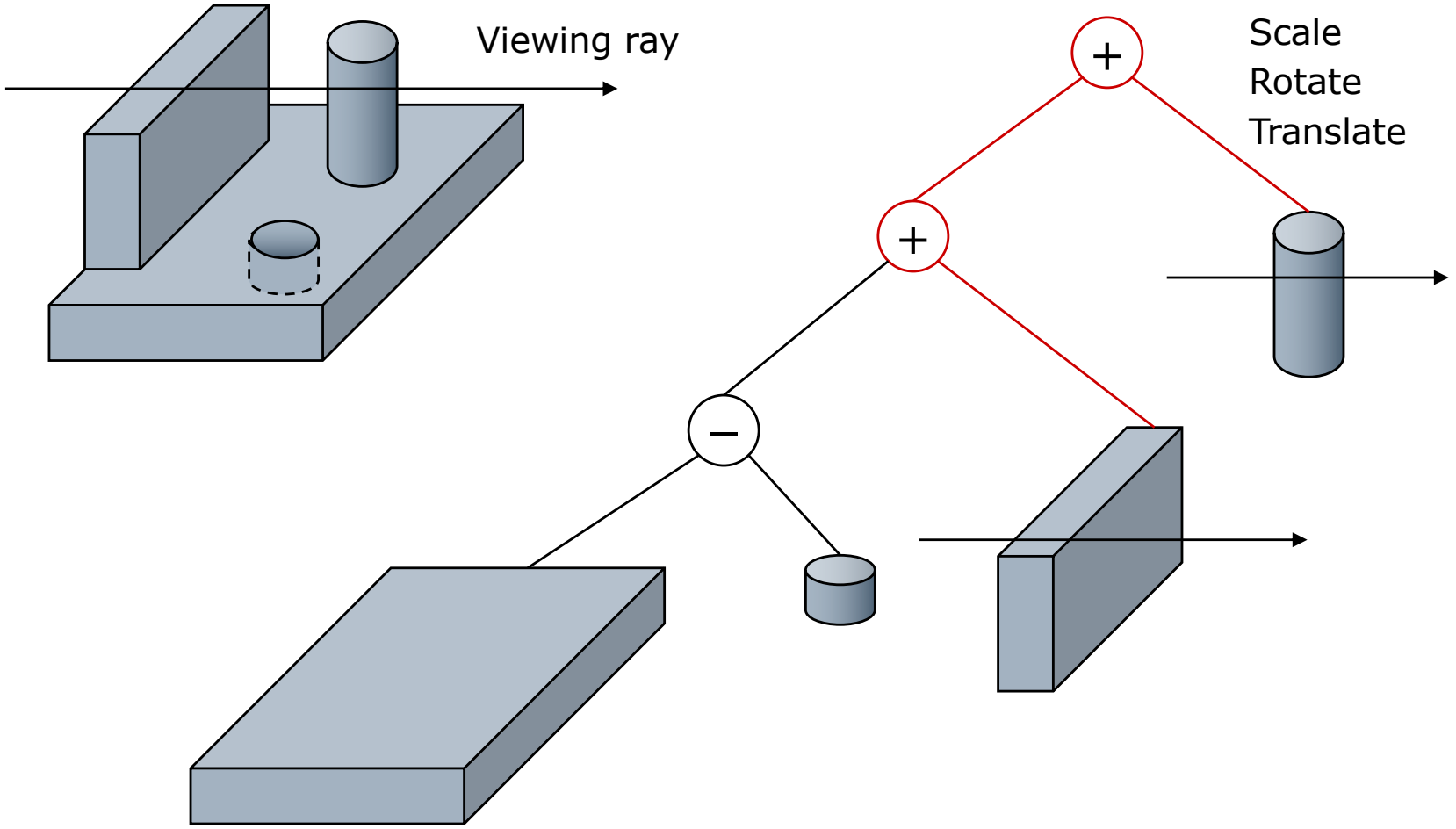


Ray primitive classification

CSG ray tracing – boolean operations



Constructive Solid Geometry (CSG)



Rendering a CSG description

- Ray tracing is the classical method that integrates into a single model:
  a. Evaluation of a boundary representation from a CSG description

```
For each view ray {

    For all CSG primitives compute the ray primitive

    Compute the global ray primitive from all the correspondent CSG ray
    primitives according with the CSG tree

    }
```

  b. Hidden surface removal – only intersection with the first primitive nearest to the viewer is considered
  c. Shading

Questions and problems

1. Why the computation of light intensity and shadows can be completed either in World or View coordinate spaces?
2. Why the determination of the hidden lines and surfaces cannot be achieved in the World coordinate space?
3. Why the rendering of parametric patch needs splitting?
4. How do you compute in the CSG ray tracing approach the visible point of an elementary graphics object component (e.g. cube or cylinder)?
5. How do you compute in the CSG ray tracing approach the visible point of the complex graphics object (e.g. made of basic geometric components)?
6. Explain the composition of a complex 3D object by the CSG binary tree, using the information associated to nodes and arcs. What is this information?